macOS Screen Recorder Review

Feature and Improvement Recommendations

Codex

May 2, 2026

Executive Summary

The current app is a clean minimal Electron screen recorder. It already has a useful foundation: display discovery, microphone and system audio toggles, microphone gain, quality presets, a countdown, recording timer, output folder selection, and a recent recordings list.

To feel like a polished macOS recording product, the next phase should focus on capture reliability, native recording controls, safer file handling, and a stronger post-recording workflow. The biggest opportunity is to evolve from a basic "start and stop" recorder into a lightweight capture studio for tutorials, demos, bug reports, and quick shareable videos.

Current Strengths

- Small, understandable Electron and TypeScript codebase.
- Secure renderer setup with `contextIsolation: true` and `nodeIntegration: false`.
- Simple IPC bridge through `preload.ts`.
- Audio mixing architecture already supports microphone and system audio streams.
- Quality presets are easy for users to understand.
- Recent recordings list gives the app a useful continuity loop.
- Build passes with the current TypeScript setup.

Highest Priority Fixes

1. Make Display Selection Match Actual Capture

The UI lets the user select a display, but the primary recording path uses `navigator.mediaDevices.getDisplayMedia()`, which still shows the system picker. That means the selected display is more of a preference than a guaranteed source.

Recommended improvement:

- Either route capture through Electron desktop sources so the selected display is enforced.
- Or update the UI copy to make it clear that macOS will ask the user to confirm the capture source.
- Add display thumbnails so users can visually confirm the intended source before recording.

2. Avoid Holding Long Recordings Fully in Memory

The renderer stores all `MediaRecorder` chunks in memory and sends the final buffer to the main process when recording stops. This can become unstable for long recordings.

Recommended improvement:

- Stream chunks incrementally to disk.
- Or send chunks to the main process every few seconds and append them to a temporary recording file.

- Show available disk space and warn before long high-quality recordings.

3. Harden File IPC

The renderer can pass file paths for saving, revealing, and deleting. For a local app this may work during development, but production builds should treat renderer input as untrusted.

Recommended improvement:

- Only allow save/delete operations inside a user-approved output folder.
- Track recording IDs in the main process instead of accepting arbitrary paths.
- Confirm delete actions in the UI.
- Prefer moving recordings to Trash instead of permanent deletion.

4. Clean Up Packaging Metadata

The app references an icon at `public/icon.icns`, but that file is not currently present in the project listing. The app also declares camera permissions and entitlements before webcam capture exists.

Recommended improvement:

- Add a real app icon and verify packaging output.
- Remove camera permission until webcam overlay is implemented.
- Replace `com.example.screenrecorder` with a real bundle identifier before distribution.
- Add signing and notarization steps for macOS releases.

Product Feature Roadmap

Capture Modes

- Full screen recording.
- Selected display recording.
- Selected window recording.
- Custom region recording with draggable crop handles.
- Fixed aspect presets such as 16:9, 4:3, 1:1, and vertical 9:16.
- Remember last capture mode and source.

Camera Overlay

- Add webcam bubble overlay for tutorials and walkthroughs.
- Support circular and rounded rectangle shapes.
- Allow drag positioning and size presets.
- Add mirror mode.
- Add camera background blur if available.
- Store overlay preferences per recording profile.

Cursor and Interaction Effects

- Show or hide cursor.
- Add click rings.

- Add cursor spotlight mode.
- Add optional mouse trail.
- Add keystroke overlay for tutorial videos.
- Let users style cursor effects per profile.

Recording Controls

- Pause and resume recording.
- Global hotkeys for start, stop, pause, and resume.
- Menu bar controller with timer and quick stop button.
- Floating recording overlay with elapsed time.
- Optional recording border around the captured area.
- Configurable countdown length.

Audio Controls

- Microphone device picker.
- System audio availability detection.
- Live mic test before recording.
- Separate system and microphone gain controls.
- Mute microphone while recording.
- Noise suppression and echo cancellation toggles.
- Optional separate audio track export for editing.

Output and Export

- MP4 export using H.264/AAC for broad compatibility.
- Keep WebM as an advanced option.
- Configurable bitrate and frame rate.
- Automatic file naming templates.
- Save location profiles.
- Optional compression after recording.
- Export presets for high quality, small file, Slack/email, YouTube, and mobile vertical clips.

Post-Recording Workflow

- Instant playback after recording.
- Trim start and end points.
- Rename recording before final save.
- Generate thumbnail preview.
- Copy file path or drag file out of the app.
- Reveal in Finder.
- Duplicate recording.
- Delete with confirmation or move to Trash.

Recording Library

- Search recordings.

- Sort by date, duration, size, or name.
- Pin favorite recordings.
- Add tags such as Demo, Bug, Tutorial, Meeting, or Draft.
- Show duration, size, resolution, frame rate, and audio sources.
- Detect missing files and offer to remove stale history entries.

UX Improvements

- Replace the tall form layout with a denser macOS-style control panel.
- Use icons for reveal, delete, rename, play, and folder actions.
- Show permission status cards for Screen Recording, Microphone, and Camera.
- Add a clear “Open System Settings” action when permission is missing.
- Make the display selector visual with thumbnails and resolution labels.
- Add a clear recording state: idle, waiting for permission, countdown, recording, paused, saving, saved, failed.
- Avoid permanent inline status messages that contain long file paths; use a compact saved state with actions.
- Add delete confirmation.
- Add empty states for recent recordings and unavailable audio.
- Make the app usable as a menu bar utility for quick screen captures.

Engineering Improvements

Reliability

- Centralize recorder state in a small state machine.
- Handle `MediaRecorder.isTypeSupported()` before choosing a codec.
- Fall back from VP9 to VP8 or another supported format.
- Handle track-ended events for every active track, not only the first video track.
- Add cleanup for partial recordings and failed saves.
- Add recovery for stale localStorage settings.

Security

- Validate IPC payloads in the main process.
- Restrict file operations to known safe directories.
- Move deletions to Trash where possible.
- Remove unused permissions.
- Keep Content Security Policy strict and remove inline styles over time.

Architecture

- Split the renderer logic into modules: settings, recording, audio, display sources, history, and UI.
- Move durable app settings to the main process or an app data file instead of relying only on localStorage.
- Store recording metadata in a JSON database under the app data directory.
- Keep UI rendering separate from recording behavior.

Testing

- Add unit tests for settings, history, filename generation, and path validation.
- Add integration tests for IPC handlers.
- Add manual QA checklist for macOS permissions.
- Add smoke tests for build and package output.

Suggested Implementation Order

1. Fix display/source selection behavior.
2. Harden IPC path validation and delete handling.
3. Add pause and resume.
4. Add global hotkeys and menu bar control.
5. Add MP4 export.
6. Add microphone device selection and audio test.
7. Add recording playback and trim.
8. Add camera overlay.
9. Add cursor effects and keystroke overlay.
10. Build the richer recording library.

Recommended App Positioning

The strongest product direction is a fast macOS capture studio for demos, tutorials, and bug reports. The app should optimize for:

- Starting a recording quickly.
- Knowing exactly what will be captured.
- Capturing clean audio.
- Stopping without losing work.
- Exporting a shareable file immediately.
- Keeping recent recordings easy to find.

That direction keeps the product focused while still leaving room for premium features such as camera overlay, cursor effects, advanced export presets, and cloud sharing later.